PLATAFORMA AI OS · TFM

AI OS Core

Manual de uso del plano de control: qué es, para qué sirve, cómo se arranca y todos los flujos que ofrece su API, con ejemplos listos para copiar.

Versión del software: **0.2.1** · Fecha: 18 de octubre de 2026
Verificado contra: AI Broker (contrato 2.11) en el PC IA, tablero de Agora, servicio de Athena, datos de Model_Drift y de Gestión Tareas IA
Suite automática: 66 pruebas en verde · Recorridos E2E reales: 31 (Broker + Agora), 33 (Model_Drift + Athena + Gestión Tareas IA) y 18 (ChatyGPT + Knowledge Orchestrator + Trading AI + Local AI Lab)

Contenido

1. Qué es AI OS Core y para qué sirve
2. Arquitectura en una página
3. Instalación y arranque
4. Convenciones de la API
5. Flujo: comprobar que el servicio está vivo
6. Flujo: metas, objetivos y auditoría
7. Flujo: registro de servicios y capacidades
8. Flujo: políticas y decisión de autorización
9. Flujo: aprobaciones humanas
10. Flujo: presupuesto y reservas
11. Flujo: bandeja de eventos y proyección canónica
12. Flujo: restore y recibos de operación
13. Flujo: grafo de recursos e identidades
14. Flujo: proyección de certificaciones
15. Flujo: paquetes de contexto
16. Flujo: planificador tipado y autonomía
17. Idempotencia de peticiones
18. Recorrido completo con el Broker y Agora
19. Configuración
20. Pruebas y verificación
21. Errores frecuentes y cómo resolverlos
22. Límites conocidos
23. Referencia rápida de endpoints
24. Glosario

1. Qué es AI OS Core y para qué sirve

AI OS Core es el **plano de control** de la plataforma AI OS: un servicio REST local (FastAPI + SQLite) que no ejecuta modelos ni agentes, sino que **decide, registra y vigila** lo que hacen los demás componentes del ecosistema del TFM: AI Broker, Model Drift, Athena, Agora, Gestión Tareas IA, Knowledge Orchestrator y ChatyGPT.

Su papel se resume en cinco verbos:

- **Orientar**: es la autoridad canónica de las *metas* (Goals) y sus *objetivos*.
- **Conocer**: mantiene el catálogo de servicios federados, sus capacidades, un grafo ligero de recursos y la correspondencia entre identidades globales y los ids nativos de cada sistema.
- **Autorizar**: evalúa políticas con lógica de tres valores, gestiona aprobaciones humanas de uso limitado y reservas de presupuesto, y combina techos de autonomía antes de dejar que un plan se despache.
- **Recordar**: recibe eventos de los demás servicios con garantías de orden y deduplicación, guarda la última foto válida de cada recurso, aparta a cuarentena lo contradictorio y conserva paquetes de contexto inmutables.
- **Recuperar**: tras un corte, dice si una operación debe reenviarse, seguirse o reconciliarse a mano, para no duplicar efectos.

> **Idea clave**
> Core es *determinista*: con las mismas entradas da siempre la misma decisión, y deja traza de cada una. Los servicios que hacen el trabajo (Broker, Agora…) le consultan antes de actuar y le informan después.

Cuándo lo usas

Situación	Qué haces en Core
Quieres que una tarea de IA cuelgue de un propósito explícito	Creas una meta y sus objetivos (§6)
Un servicio nuevo se une al ecosistema	Lo registras con sus capacidades; Core rechaza URLs peligrosas (§7)
Antes de lanzar algo con coste o datos sensibles	Evalúas políticas (§8), pides aprobación (§9) y reservas presupuesto (§10)
Un servicio cambia el estado de algo	Envía un evento a la bandeja; Core mantiene la versión canónica (§11)
Se cortó la luz a mitad de una operación	Consultas el estado de la operación antes de reenviar (§12)
Un planificador propone varias tareas	Registras la propuesta y pides la decisión de despacho con los techos de autonomía (§16)

2. Arquitectura en una página

API (FastAPI)
`api/routes/*`: un router por dominio, validación Pydantic, formato de error único y middleware de idempotencia.

Aplicación
`application/*_service.py`: casos de uso (crear meta, evaluar políticas, decidir plan…).

Dominio
`domain/*`: entidades puras e invariantes (precedencia de decisiones, lógica ternaria, niveles A0–A5).

Repositorios
`infrastructure/db/repositories`: SQL explícito, transacciones `BEGIN IMMEDIATE`.

SQLite WAL
Un único fichero, `synchronous=FULL`, migraciones sólo hacia delante, aplicadas al arrancar.

Seguridad de red
`infrastructure/http/security_client.py`: validación SSRF y allowlist de hosts.

Core **no hace llamadas salientes**: el registro de un servicio valida su URL pero no la visita. Los demás servicios hablan con Core y no al revés. Esto lo mantiene arrancable "con cero dependencias", como exige la especificación (P01).

3. Instalación y arranque

Requisitos

- Python 3.11 o superior (verificado con el Python 3.14 del sistema).
- Paquetes: `fastapi`, `uvicorn[standard]`, `pydantic` v2, `pydantic-settings`; para pruebas, `pytest` y `httpx`.

```
cd "D:\Desarrollo\Proyectos TFM\AI_OS_core"
python -m pip install -e ".[dev]"
```

Con el lanzador (recomendado)

Doble clic en `start.bat` (o `run.bat`) abre un menú. Cada opción también acepta un argumento para usarse sin menú:

Opción	Argumento	Qué hace
1	`app` / `api`	Arranca el servicio en `http://127.0.0.1:8000` con recarga automática
2	`tests`	Ejecuta la suite completa con pytest (si falla, reintenta con unittest)
3	`migrate`	Aplica migraciones pendientes a la base real y lista las versiones aplicadas
4	`smoke`	Pruebas de humo de los endpoints básicos (en una base temporal)
5	`docs`	Abre Swagger UI en el navegador y arranca el servicio
6	`exit`	Sale

```
run.bat app        :: servicio
run.bat tests      :: suite completa
run.bat migrate    :: "Migraciones completadas. Versiones aplicadas: []" si ya está al día
```

A mano

```
set PYTHONPATH=D:\Desarrollo\Proyectos TFM\AI_OS_core\src
python -m uvicorn ai_os_core.app:app --host 127.0.0.1 --port 8000
```

Al arrancar, Core crea la carpeta de datos si no existe y aplica las migraciones antes de aceptar peticiones. La documentación interactiva queda en `http://127.0.0.1:8000/docs` (Swagger) y `/redoc`.

> **¿Dónde están mis datos?**
>
> `%LOCALAPPDATA%\AIOSCore\data\ai_os_core.db` (más los ficheros `-wal` y `-shm`). Para trabajar con otra base, define `AI_OS_DB_PATH` antes de arrancar. Para copiar la base con el servicio en marcha, detenlo primero o copia los tres ficheros juntos.

4. Convenciones de la API

- Prefijo común `/api/v1` salvo `/health` y `/ready`. Cuerpos y respuestas en JSON UTF-8.
- Las fechas son ISO 8601. Core las guarda en UTC; acepta `Z` o desfases (`+02:00`) y rechaza textos que no sean fechas.
- Si no das un identificador, Core lo genera con prefijo: `goal_…`, `obj_…`, `resv_…`, `ctx_…`, `plan_…`.
- Los listados paginados aceptan `limit` (1–100, por defecto 50) y `offset`, y devuelven `{items, total_returned, limit, offset}` con orden estable.

Autenticación

Si arrancas Core con la variable `AI_OS_API_TOKEN`, toda la API exige la cabecera `Authorization: Bearer <token>`; sin ella o con otro valor responde **401**. `/health`, `/ready` y la documentación (`/docs`, `/redoc`, `/openapi.json`) siguen públicas para que un lanzador sepa si hay algo escuchando. Sin la variable, la API queda abierta como en 0.2.0. Una petición rechazada por token no llega a reservar su clave de idempotencia.

Un token por app: con `AI_OS_API_TOKENS="chatygpt=token1;athena=token2"` cada token identifica a su app. Si esa app declara ser otra, en la cabecera `X-Principal-Id` o en el campo `principal_id` del cuerpo, Core responde **403** `PrincipalMismatch`; si no declara nada, Core usa el nombre del token, también para separar las claves de idempotencia. Puede convivir con `AI_OS_API_TOKEN`.

```
set AI_OS_API_TOKEN=pon-aqui-un-secreto-largo
run.bat app
curl http://127.0.0.1:8000/api/v1/goals -H "Authorization: Bearer pon-aqui-un-secreto-largo"
```

Códigos de estado

Código	Significado en Core
200 / 201	Lectura correcta / recurso creado
400	Valor inválido: estado, fecha, operador, nivel, URL bloqueada por SSRF, referencia a un padre inexistente, entero que no cabe en 64 bits, decisión ALLOW no emitida por Core
401	Falta el token de Core o no es válido (sólo con `AI_OS_API_TOKEN` definido)
403	Token de una app que declara ser otra (`PrincipalMismatch`). Denegado: aprobación caducada, revocada, agotada o de otro digest; principal sin scope o sin clearance suficiente
404	No existe lo que pides
409	Choca con lo ya registrado: id repetido, revisión obsoleta, saldo insuficiente, reserva ya consumida, clave de idempotencia reutilizada o en curso, recibo que retrocede, dato de otro propietario, `event_id` reutilizado con otro contenido
422	Falta un campo obligatorio o tiene el tipo equivocado (validación del esquema)

Formato de error

```
{
  "error":  { "type": "Conflict", "message": "Goal 'g1' ya existe.", "details": {} },
  "detail": "…"        // en errores HTTP genéricos y en los 422 (lista de FastAPI), por compatibilidad
}
```

Desde 0.2.1 los **422** de validación usan el mismo sobre: `error.type = "RequestValidationError"`, `error.message` con campo y motivo, y `detail` conserva la lista original.

Los ejemplos de este manual usan `curl`. En PowerShell, sustituye por `Invoke-RestMethod -Method Post -Uri … -ContentType 'application/json' -Body '…'`, o usa Swagger UI.

5. Flujo: comprobar que el servicio está vivo

`GET` `/health` responde siempre que el proceso viva; `GET` `/ready` además abre la base y devuelve la versión de esquema (503 si no puede).

```
curl http://127.0.0.1:8000/health
{"status":"healthy","service":"ai_os_core","version":"0.2.1"}

curl http://127.0.0.1:8000/ready
{"status":"ready","service":"ai_os_core","schema_version":2}
```

6. Flujo: metas, objetivos y auditoría

Una **meta** (Goal) es el "para qué" de alto nivel; los **objetivos** la descomponen. Toda tarea o plan del ecosistema debería poder colgarse de una meta. Cada cambio de estado incrementa la `revision` y queda auditado.

Estados válidos: `pending`, `active` (inicial), `paused`, `completed`, `failed`, `cancelled`.

1. **Crear la meta** `POST` `/api/v1/goals`

   ```
   curl -X POST http://127.0.0.1:8000/api/v1/goals -H "Content-Type: application/json" -d "
   {\"goal_id\":\"goal_tfm\",\"title\":\"Entregar el TFM\",\"description\":\"Memoria y
   defensa\",\"owner_service\":\"gestion_tareas\"}"
   ```

   Campos: `title` y `description` obligatorios; `goal_id`, `owner_service` (por defecto `ai_os_core`) y `parent_goal_id` opcionales. Repetir un `goal_id` existente da **409** (nunca sobrescribe); un padre inexistente da **400**.
2. **Añadir objetivos** `POST` `/api/v1/objectives` con `{"goal_id":"goal_tfm","title":"Redactar capítulo 3"}`.
3. **Consultar**: `GET` `/api/v1/goals/{id}`, `GET` `/api/v1/goals?limit=50&offset=0`, `GET` `/api/v1/goals/{id}/objectives`, `GET` `/api/v1/objectives/{id}`.
4. **Cambiar estado** `PATCH` `/api/v1/goals/{id}/status` o `/api/v1/objectives/{id}/status` con `{"status":"completed","principal_id":"usuario"}`. La revisión sube en uno.
5. **Ver la traza** `GET` `/api/v1/goals/{id}/audit`: lista append-only de creación y cambios, con quién y cuándo.

7. Flujo: registro de servicios y capacidades

Core mantiene el catálogo de servicios del ecosistema y de lo que cada uno sabe hacer. Al registrar, valida la URL contra ataques SSRF; no la visita.

1. **Registrar (o actualizar) un servicio** `POST` `/api/v1/services`

   ```
   { "service_id": "ai_broker", "name": "AI Broker", "base_url": "http://192.168.1.52:8765",
     "version": "2.11", "health_status": "healthy", "capabilities": ["llm.task"],
     "metadata": { "auth_header": "X-Admin-Token" } }
   ```

   Repetir el mismo `service_id` actualiza el registro (es idempotente). `health_status`: `healthy`, `degraded`, `down`, `unknown`; cualquier otro valor → 400 (antes se convertía en `healthy` sin avisar). `service_id` y `name` no pueden ir vacíos.
2. **Declarar capacidades** `POST` `/api/v1/capabilities` con `capability_id`, `name`, `description`, `provider_service_id` y opcionalmente `input_schema` / `output_schema`. El proveedor debe estar registrado (si no, 400).
3. **Consultar**: `GET` `/api/v1/services`, `/api/v1/services/{id}`, `/api/v1/capabilities/{id}`.

Qué URLs acepta el registro

Se admite	Se rechaza (400)
`localhost`, `127.0.0.1`, `::1` y cualquier loopback Redes privadas de la LAN (`192.168.x.x`, `10.x`, `172.16–31.x`) mientras `AI_OS_ALLOW_PRIVATE_NETS=true` Nombres DNS añadidos con `AI_OS_ALLOWED_HOSTS`	Esquemas distintos de http/https (`file:`, `ftp:` …) Metadatos de nube (`169.254.169.254`) y todo link-local (`169.254.0.0/16`, `fe80::/10`), también disfrazados como IPv6 (`::ffff:169.254.169.254`) `0.0.0.0`, multicast y rangos reservados IPs públicas y nombres DNS fuera de la allowlist

8. Flujo: políticas y decisión de autorización

Una **política** dice: "en este *scope*, si se cumplen estas condiciones, la decisión es X". Core evalúa todas las políticas habilitadas del scope (y las de scope `*`) contra un **contexto** y devuelve una decisión consolidada.

Lógica de tres valores

Cada condición da `TRUE`, `FALSE` o `UNKNOWN`. Un dato ausente, nulo o no comparable (un texto contra `greater_than`) es `UNKNOWN`: Core nunca adivina.

Operador	Uso	Ausente / null
`exists`	El campo está en el contexto (aunque sea null)	FALSE si ausente
`equals`, `not_equals`	Igualdad. `equals` con `value: null` pregunta "¿es null?"	UNKNOWN
`greater_than`, `less_than`, `greater_or_equal`, `less_or_equal`	Comparación numérica u ordenable	UNKNOWN
`in`	Pertenencia; `value` debe ser una lista	UNKNOWN

`condition_mode`: `all` (algún FALSE → FALSE; si no, algún UNKNOWN → UNKNOWN) o `any` (algún TRUE → TRUE; si no, algún UNKNOWN → UNKNOWN). Una política sin condiciones sólo es válida con `unconditional: true`. Operadores o modos desconocidos se rechazan al crear (400).

Combinación

- Precedencia estricta: **DENY > REQUIRE_APPROVAL > INDETERMINATE > ALLOW**.
- Sin ninguna política en el scope → **DENY** (denegar por defecto).
- Una política restrictiva que no se puede evaluar aporta **INDETERMINATE**: un "DENY si coste > 5" sin coste no queda tapado por un ALLOW.
- Restricciones `max_*` se quedan con la más baja y `min_*` con la más alta; si un mínimo supera a su máximo → DENY. Las `obligations` se acumulan.
- `executable` sólo es `true` con ALLOW y sin incógnitas de seguridad (campos cuyo nombre contiene *security* o *auth*); una aprobación humana no resuelve esas incógnitas.

1. **Crear políticas** `POST` `/api/v1/policies`

```
{ "policy_id": "pol_conf", "policy_key": "confidencial_requiere_aprobacion", "scope": "broker:invoke",
  "decision_if_matched": "REQUIRE_APPROVAL",
  "conditions": [ { "field": "data_classification", "operator": "in", "value":
["confidential","local_only"] } ] }

{ "policy_id": "pol_cost", "policy_key": "coste_maximo", "scope": "broker:invoke",
  "decision_if_matched": "DENY",
  "conditions": [ { "field": "estimated_cost", "operator": "greater_than", "value": 5 } ] }

{ "policy_id": "pol_base", "policy_key": "auditar_siempre", "scope": "broker:invoke",
  "decision_if_matched": "ALLOW", "unconditional": true,
  "obligations": ["audit_trace"], "constraints": { "max_output_tokens": 600 } }
```

   Reenviar el mismo `policy_id` la actualiza. `enabled: false` la desactiva sin borrarla.

2. **Evaluar** `POST` `/api/v1/policies/evaluate`

```
{ "scope": "broker:invoke", "principal_id": "chatygpt",
  "context": { "data_classification": "internal", "estimated_cost": 0.5 } }
→ { "authorization": "ALLOW", "constraints": {"max_output_tokens": 600},
    "obligations": ["audit_trace"], "matched_policy_refs": ["pol_base"],
    "unknown_inputs": [], "executable": true, "trace_id": "trace_a19e27f268a6" }
```

Con `confidential` el resultado es REQUIRE_APPROVAL; sin `estimated_cost`, INDETERMINATE con `unknown_inputs: ["estimated_cost"]`. Cada evaluación deja una traza de decisión con el hash del contexto; su `trace_id` viaja en la respuesta y es lo que el planificador exige para aceptar un ALLOW (sección 16).

3. **Revisar** `GET` `/api/v1/policies` o `/api/v1/policies?scope=broker:invoke`.

9. Flujo: aprobaciones humanas

Una **autorización** es un permiso concedido (normalmente por una persona) con un número máximo de usos, caducidad opcional y, si se quiere, ligado a una acción exacta mediante su digest. Es inmutable: no se edita, se revoca.

1. **Conceder** `POST` `/api/v1/approvals`

```
{ "authorization_id": "auth_publicar_42", "scope": "broker:invoke", "max_uses": 1,
  "expires_at": "2026-10-08T18:00:00+01:00", "expected_digest": "9f3c…", "granted_by": "usuario" }
```

   `max_uses` ≥ 1. Con `expected_digest` la autorización es *exact_action*: sólo vale para la operación cuyo hash coincide. Sin él es *scoped*: vale para cualquier operación del scope.

2. **Consumir un uso** justo antes de actuar `POST` `/api/v1/approvals/use`

```
{ "use_id": "use_1", "authorization_id": "auth_publicar_42",
  "operation_id": "op_publicar_42", "operation_digest": "9f3c…" }
→ {"status":"committed","use_id":"use_1"}
```

   Reglas: caducada, revocada, agotada o digest distinto del aprobado → **403**. Repetir el mismo `operation_id` con el mismo digest es un reintento legítimo y **no gasta** otro uso; con distinto digest → **409**. Si dos operaciones compiten por el último uso, sólo una lo consigue (verificado con 10 peticiones simultáneas).

3. **Ver el estado** `GET` `/api/v1/approvals/{id}/status` → `status` (`active`, `consumed`, `expired`, `revoked`), `used_count`, `remaining_uses`.

4. **Revocar** `POST` `/api/v1/approvals/{id}/revoke` con `{"revoked_by":"usuario","reason":"…"}`. Revocar dos veces no cambia nada.

10. Flujo: presupuesto y reservas

Antes de una operación con coste se **reserva** saldo; al terminar se **consume** (o se **libera** si no se hizo). Así dos operaciones simultáneas no pueden gastar el mismo dinero.

1. **Abrir cuenta** `POST` `/api/v1/budget/accounts` con `{"account_id":"broker_cloud","balance":20}` (409 si ya existe).
2. **Reservar** `POST` `/api/v1/budget/reserve` con `{"account_id":"broker_cloud","amount":0.5,"reservation_id":"resv_1","expires_at":"…"}`. Saldo disponible insuficiente → **409**; cuenta inexistente → 404; `reservation_id` repetido → 409.
3. **Consumir** `POST` `/api/v1/budget/consume` con `{"reservation_id":"resv_1"}`: descuenta del saldo. Consumir dos veces → 409.
4. **o Liberar** `POST` `/api/v1/budget/release`: devuelve el importe al disponible. Liberar una consumida → 409; una inexistente → 404; dos veces → sin efecto.
5. **Consultar**: `GET` `/api/v1/budget/accounts/{id}` → `balance`, `reserved`, `available`; `GET` `/api/v1/budget/reservations/{id}` → estado `reserved`, `consumed`, `released` o `expired`.

> **Caducidad**
> Una reserva con `expires_at` vencido y sin consumir se marca `expired` y su importe vuelve al disponible en cuanto se toca la cuenta: al reservar y también al **consultar** la cuenta o la reserva (antes las consultas seguían mostrando el saldo retenido). Una reserva cuyo `expires_at` ya pasó al crearla → 400.

11. Flujo: bandeja de eventos y proyección canónica

Los servicios informan a Core de los cambios mediante **eventos**. Cada productor numera sus eventos por flujo (`producer_id` + `stream_id` + `stream_epoch`) con `source_sequence` consecutiva. Core guarda todos los eventos y mantiene la **proyección canónica** de cada recurso: el último `full_state` válido de mayor revisión.

`event_type`	Efecto
`full_state`	Foto completa del recurso. Es el único tipo que cambia la proyección canónica.
`delta`, `audit`, `notification`	Se guardan y cuentan para la secuencia, pero no cambian la proyección.

1. **Enviar** `POST` `/api/v1/events/inbox`

```
{ "event_id": "ev_001", "producer_id": "ai_broker", "stream_id": "tasks", "stream_epoch": 1,
  "source_sequence": 1, "event_type": "full_state", "resource_id": "broker_task:abc",
  "resource_type": "broker_task", "revision": 2, "payload": {"status": "completed"},
  "state_digest": "sha256-del-estado" }
→ { "event_id": "ev_001", "accepted_in_canonical_projection": true,
    "quarantine": false, "duplicate": false, "reason": null }
```

2. **Leer la versión vigente** `GET` `/api/v1/events/canonical/{resource_id}` → revisión, payload, digest y fecha.
3. **Revisar incidencias**: `GET` `/api/v1/events/gaps` (saltos de secuencia sin resolver) y `GET` `/api/v1/events/quarantine` (eventos apartados).

Qué pasa en cada caso

Caso	Respuesta de Core
Reentrega del mismo `event_id` (reintento de red)	200 con `duplicate: true`; no se reprocesa.
Mismo `event_id` con otro contenido (payload, revisión, digest…)	409: no es una reentrega, es divergencia.
Otro evento con una secuencia ya ocupada	409.
Salto de secuencia (llega la 5 tras la 1)	Se registra un REPLAY_GAP y el evento va a cuarentena (C05). Mientras falte la 2, todo lo posterior también espera en cuarentena.
Llega por fin la secuencia que faltaba	Se aplica y Core **reanuda** en orden los eventos que esperaban; el REPLAY_GAP pasa a resuelto y desaparece de `/events/gaps`.
Evento de un `stream_epoch` anterior al vigente	Se guarda sólo para auditoría (`reason` `STALE_EPOCH`); no toca la proyección.
`source_sequence` < 1 o `stream_epoch` < 0	422.
Dos `full_state` de la misma revisión con distinto `state_digest`	El segundo va a cuarentena (C03).
`full_state` tardío de revisión menor	Se guarda para auditoría pero la proyección no retrocede (C02).

12. Flujo: restore y recibos de operación

Tras una caída, la pregunta es "¿reenvío o no?". Los receptores registran **recibos** del estado de cada operación y Core traduce ese estado en una acción recomendada.

1. **El receptor informa** `POST` `/api/v1/operations/receipts` con `{"operation_id":"op_42","receiver_service":"ai_broker","status":"in_progress","evidence":{"task_id":"task_…"}}`. Se puede reenviar con el estado nuevo, pero el estado **sólo avanza** (`not_seen` →

`accepted` → `in_progress` → terminal): retroceder, cambiar un estado terminal (`succeeded`, `failed_terminal`, `cancelled`) o que otro `receiver_service` reescriba el recibo → **409**. Repetir el mismo estado es idempotente.

2. **Quien va a reenviar consulta** `POST` `/api/v1/operations/status` con `{"operation_id":"op_42","target_service_id":"ai_broker"}`.

Estado del recibo	`recommended_action`	Qué hacer
sin recibo (`not_seen`)	`retry_allowed`	Reenviar con el mismo `operation_id`, tras revalidar política y aprobación
`accepted`, `in_progress`	`follow_up_no_resend`	No reenviar; seguir consultando
`succeeded`	`reconstruct_state_no_resend`	Reconstruir el estado local a partir del resultado
`failed_terminal`, `cancelled`	`do_not_resend_without_new_decision`	Sólo con una decisión/operación nueva
`unknown`	`manual_reconciliation_required`	Revisión humana

13. Flujo: grafo de recursos e identidades

El **grafo de recursos** es un mapa ligero de qué existe en el ecosistema y cómo se relaciona (una tarea "ejecutada por" un trabajo de Agora, un objetivo "cumplido por" una tarea del Broker). Los **identity bindings** traducen un id global de AI OS al id nativo de cada sistema.

1. **Proyectar un recurso** `PUT` `/api/v1/resources` con `{"resource_id":"task_77","resource_type":"task","owner_service":"gestion_tareas","revision":3,"data":{…}}`. El propietario no puede cambiar y la revisión no puede bajar (409 en ambos casos).
2. **Relacionar** `POST` `/api/v1/resources/relations` con `{"source_id":"obj_1","target_id":"task_77","relation_type":"fulfilled_by"}`. Ambos deben existir; repetir no duplica.
3. **Consultar** `GET` `/api/v1/resources/{id}`: el recurso y sus relaciones salientes.
4. **Vincular identidades** `PUT` `/api/v1/identity-bindings` con `{"global_id":"op_42","native_id":"task_9f2c","source_system":"ai_broker","adapter_revision":1}`; resolver con `GET` `/api/v1/identity-bindings/{global_id}/{source_system}`. Una `adapter_revision` menor que la registrada → 409.

14. Flujo: proyección de certificaciones

Model Drift evalúa si un modelo está certificado para una capacidad. Core guarda una **proyección** de ese veredicto para poder consultarlo al decidir, sin llamar a Model Drift.

1. **Registrar** `POST` `/api/v1/certifications`

```
{ "model_id": "gemma4:12b", "capability_id": "llm.task", "version": "1",
  "source_system": "model_drift", "evaluation_verdict": "CERTIFIED",
  "is_applicable": true, "revision": 5 }
```

   Veredictos: `CERTIFIED`, `REJECTED`, `INDETERMINATE`, `EXPIRED`. Una revisión igual o mayor actualiza; una menor (llegada tarde) → **409** y la proyección no cambia. Sólo el `source_system` que la registró puede reescribirla (otro → 409).

2. **Consultar** `GET` `/api/v1/certifications` o `/api/v1/certifications/{model_id}/{capability_id}/{version}`. El `model_id` puede llevar barras, como los modelos del Broker: `/api/v1/certifications/ollama/local/muse-glimmer:latest/suite:facturacion/1`.

15. Flujo: paquetes de contexto

Un **ContextPackage** fija, de forma inmutable, el contexto exacto con que se tomó una decisión o se lanzó una tarea, para poder reproducirlo después aunque las fuentes cambien. Su `snapshot_hash` es un SHA-256 determinista del contenido.

1. **Crear** `POST` `/api/v1/context/packages`

```
{ "goal_id": "goal_tfm", "classification_level": "INTERNAL",
  "resources": [ { "resource_id": "obj_1", "resource_type": "objective", "owner_service": "ai_os_core"
} ],
  "payload": { "prompt_digest": "9f3c…" },
  "principal_id": "athena", "principal_role": "agent",
  "principal_clearance": "INTERNAL", "principal_scopes": ["context:build"] }
```

   Niveles: `PUBLIC` < `INTERNAL` < `CONFIDENTIAL` < `RESTRICTED`. Sin el scope `context:build`, o pidiendo un nivel por encima del clearance del principal → **403**. Niveles ilegibles → 400.

2 **Recuperar** por id `GET` `/api/v1/context/packages/{id}` o por hash `GET` `/api/v1/context/snapshots/{hash}`.

16. Flujo: planificador tipado y autonomía

Un planificador (persona o agente) propone un **plan tipado**: una lista de tareas asignadas a servicios, colgado de una meta, con un nivel de autonomía solicitado y la decisión de política que lo respalda. Core lo guarda y, cuando se pide, decide si puede **despacharse** combinando los techos de autonomía aplicables.

Nivel	Significado
A0	Totalmente supervisado: nunca se despacha solo
A1	Asistido, confirmación en cada paso
A2	Semiautónomo con aprobación de inicio
A3	Autónomo condicionado a políticas y presupuesto
A4	Alta autonomía con auditoría posterior
A5	Totalmente autónomo dentro del sandbox

1. **Proponer** `POST` `/api/v1/planner/proposals`

```
{ "goal_id": "goal_tfm", "requested_autonomy": "A3",
  "policy_decision": { …respuesta íntegra de /policies/evaluate, con su trace_id… },
  "tasks": [
    { "task_id": "resumir", "title": "Resumir fuentes", "description": "Tarea LLM",
      "assigned_service": "ai_broker", "requires_human_approval": false, "estimated_duration_sec": 120
},
    { "task_id": "publicar", "title": "Publicar", "description": "Acción irreversible",
      "assigned_service": "agora", "requires_human_approval": true } ],
  "principal_id": "athena", "principal_scopes": ["planner:propose"] }
```

   La meta debe existir (404), hace falta al menos una tarea con `task_id` únicos (400) y el principal necesita el scope `planner:propose` (403). Un `ALLOW` sólo se acepta si trae el `trace_id` de una evaluación real de Core y coincide con ella (resultado y `executable`); uno escrito a mano o alterado → **400**.

2. **Decidir** `POST` `/api/v1/planner/proposals/{id}/decision` con `{"applicable_ceilings":["A3","A4"]}`.

```
→ { "approved_for_dispatch": true, "effective_autonomy": "A3",
    "reasons": ["Aprobado para despacho con nivel efectivo A3."] }
```

Reglas de la decisión

- La autonomía efectiva es el **mínimo** entre la solicitada y todos los techos. Ninguna capa la eleva.
- Un techo ilegible (p. ej. `"A1x"`) → 400: ignorarlo subiría la autonomía.
- No se despacha si no hay decisión de política, si no es ALLOW, si no es ejecutable, si el nivel efectivo es A0, o si alguna tarea requiere aprobación humana y el nivel es A0–A1. Todas las razones aparecen en `reasons`.

> **Importante**
> Core decide si un plan *puede* despacharse; no lo despacha. Crear las tareas en Gestión Tareas IA o las tarjetas en Agora es responsabilidad de quien pidió la decisión.

17. Idempotencia de peticiones

Cualquier petición POST/PUT/PATCH puede hacerse segura frente a reintentos añadiendo la cabecera `X-Idempotency-Key` (o `Idempotency-Key`). Opcionalmente: `X-Principal-Id` (quién llama) y `X-Operation-Id`.

- La clave vale dentro de su espacio **principal + método y ruta + clave**. La misma clave usada por otro principal o en otra ruta es otra operación.

- Misma clave y mismo cuerpo → se devuelve la respuesta original con `X-Idempotent-Replay: true`; no se repite el efecto.
- Misma clave y cuerpo distinto → **409** `IdempotencyConflict`.
- Misma clave mientras la primera petición aún se procesa → **409** `IdempotencyInProgress` con `Retry-After: 1`.
- Si la operación falla con error 5xx, la clave se libera y puede reintentarse. Las respuestas 4xx sí quedan registradas.

```
curl -X POST http://127.0.0.1:8000/api/v1/goals -H "Content-Type: application/json" ^
     -H "X-Idempotency-Key: alta-meta-tfm" -H "X-Principal-Id: gestion_tareas" ^
     -d "{\"title\":\"Entregar el TFM\",\"description\":\"…\"}"
```

18. Recorrido completo con el Broker y Agora

Este es el flujo que se ejecutó de verdad el 7 de octubre de 2026 (y de nuevo el 18 de octubre con 0.2.1) contra el AI Broker del PC IA (`http://192.168.1.52:8765`, contrato 2.11) y el tablero de Agora de este equipo (`https://127.0.0.1:8741`). Superó 31 comprobaciones. Sirve como plantilla para integrar cualquier cliente.

1. **Comprobar Core**: `/health` y `/ready`.
2. **Registrar los servicios reales**: `ai_broker` (IP privada de la LAN, admitida) y `agora_board` (loopback https), y la capacidad `llm.task`. Se comprueba que una capacidad de un proveedor inexistente se rechaza.
3. **Crear meta y objetivo** para la tarea.
4. **Definir y evaluar políticas** del scope `broker:invoke`: datos internos y baratos → ALLOW ejecutable con obligación `audit_trace` y tope de 600 tokens; datos confidenciales → REQUIRE_APPROVAL; sin coste → INDETERMINATE.
5. **Reservar presupuesto** (0,5 sobre una cuenta de 2).
6. **Calcular el digest** del cuerpo exacto que se enviará al Broker, conceder una aprobación *exact_action* de un uso con ese digest y consumirla. Un digest manipulado se rechaza con 403.
7. **Fijar el contexto** en un ContextPackage y **registrar el plan** (Broker + Agora); la decisión sale aprobada en A3. Un plan con un ALLOW no emitido por Core se rechaza.
8. **Restore previo**: la operación aún no existe → `retry_allowed`; se registra el recibo `accepted`.
9. **Invocar al Broker** (`POST /api/v1/tasks` con cabecera `X-Admin-Token`, clave de idempotencia estable y `risk.data_classification` tomado del contexto evaluado). Recibo `in_progress` con el `task_id` → `follow_up_no_resend`. Sondeo hasta `completed`.
10. **Cerrar**: recibo `succeeded` con el hash del resultado → `reconstruct_state_no_resend`; eventos `full_state` de la tarea (un evento tardío no hace retroceder la proyección); consumo del presupuesto; relación objetivo → tarea en el grafo; identity binding operación → `task_id`; objetivo a `completed`. Un recibo `not_seen` tardío ya no puede reabrir la operación (409).
11. **Idempotencia del Broker**: reenviar la misma petición devuelve la misma tarea, no una nueva.
12. **Agora (sólo lectura)**: se lee el tablero y los perfiles con el token Bearer del tablero y se proyecta una foto en Core como `full_state` de `agora:board`.

Respuesta real obtenida del Broker: *"Un plano de control sirve para coordinar y gestionar los recursos y operaciones del sistema distribuido, asegurando su correcto funcionamiento y comunicación entre sus componentes."*

> **Cómo repetirlo**
>
> El guion es `python scripts\e2e_broker_agora.py <url_core> <token_broker> <token_agora>`. Necesita el PC IA encendido, el tablero de Agora arrancado (`C:\Agora\Agora Tablero.cmd`) y el certificado `C:\Agora\pki\ca.crt`. El Broker cambia su token en cada arranque: si da 403, pide el token nuevo. En Agora no se crean tarjetas, porque eso encargaría trabajo real al PC IA.

Segundo recorrido: Model_Drift, Athena y Gestión Tareas IA

Ejecutado el 18 de octubre de 2026 con Core protegido por token (33 comprobaciones): `python scripts\e2e_ecosistema.py <url_core> <url_athena> <token_athena> <model_drift.sqlite> <copia_gti.sqlite3> <carpeta_trabajo>`.

1. **Model_Drift → certificaciones**: se leen (sólo lectura) sus resultados de `pass_rate` y se proyectan como certificaciones. Regla del recorrido: media ≥ 0,8 con al menos 3 casos aplicables → CERTIFIED; < 0,5 → REJECTED; si no, INDETERMINATE. Resultado real: `muse-glimmer:latest` CERTIFIED, `lfm2:24b` REJECTED, el resto INDETERMINATE. Una política "sólo modelos certificados" autoriza el primero y niega los demás; otra app no puede reescribir esos veredictos.

2. **Athena** (servicio en `127.0.0.1:8771`, sobre el Broker real): Core evalúa su política (sólo lectura → ALLOW; escritura → REQUIRE_APPROVAL; `exec=allow` → DENY), reserva presupuesto, aprueba el plan y **elige el modelo** entre los que ofrece Athena según las certificaciones: `muse-glimmer:latest` sí, `qwen3.8:27b` y `lfm2:24b` no. El run (perfil `questions`) se lanza con clave de idempotencia, sus estados entran en el inbox y termina `completed`; el historial de Athena confirma que respondió el modelo elegido y que leyó las 4 líneas del fichero sin modificarlo.
3. **Gestión Tareas IA**: desde una **copia** de su base, el proyecto pasa a meta y las 76 tareas a recursos y eventos `full_state`; el estado canónico de las 76 coincide con el origen y un cambio posterior actualiza la proyección.

Tercer recorrido: ChatyGPT, Knowledge Orchestrator, Trading AI y Local AI Lab

`python scripts\e2e_resto_apps.py <url_core> <carpeta_con_copias>` (18 comprobaciones, verde el 18 de octubre con token común y con tokens por app). Usa **copias** de sus bases abiertas en sólo lectura.

- **ChatyGPT**: sus 4 293 tareas reales al Broker entran como recibos. Core recomienda no reenviar las 4 277 completadas, no reenviar sin nueva decisión las 11 fallidas y la cancelada, y sí reenviar las 4 que nunca llegaron al Broker. Un recibo tardío no reabre una completada. Sus 34 conversaciones se proyectan idénticas.
- **Knowledge Orchestrator**: sus 235 tareas al Broker, incluidas 14 en cola y 3 en curso, para las que Core manda seguir sin reenviar. Sus 122 flujos se proyectan idénticos.
- **Trading AI desktop**: 11 backtests proyectados; reenviar uno con otra huella en la misma revisión va a cuarentena y la proyección no cambia.
- **Local AI Lab**: 10 registros de experimentos proyectados idénticos.
- **LARM** no entra: hoy sólo es especificación y prompts, no hay programa que gobernar.

> **Lo que esto no prueba**
> Ninguna app llama hoy a Core por su cuenta: en los dos recorridos el guion hace de cliente. Conectar una app es trabajo en esa app. La API de Athena no expone el texto final de su respuesta, sólo la evidencia (ficheros leídos, modelo, verificación).

19. Configuración

Variable de entorno	Por defecto	Para qué
`AI_OS_DB_PATH`	`%LOCALAPPDATA%\AIOSCore\data\ai_os_core.db`	Fichero SQLite (`:memory:` para pruebas)
`AI_OS_SQLITE_SYNCHRONOUS`	`FULL`	Durabilidad de SQLite
`AI_OS_SQLITE_TIMEOUT_MS`	`5000`	Espera máxima ante bloqueo de escritura
`AI_OS_API_TOKEN`	(vacío: API abierta)	Token Bearer que exige toda la API salvo `/health`, `/ready` y la documentación
`AI_OS_API_TOKENS`	(vacío)	Un token por app: `app=token;otra=token2`. Fija el principal de cada llamada
`AI_OS_ALLOWED_HOSTS`	(vacío)	Hosts DNS extra admitidos en el registro, separados por comas
`AI_OS_ALLOW_PRIVATE_NETS`	`true`	Admitir IPs de redes privadas al registrar servicios
`AI_OS_ENV`, `AI_OS_HOST`, `AI_OS_PORT`	`development`, `127.0.0.1`, `8000`	Informativas; el puerto real lo fija uvicorn

20. Pruebas y verificación

```
run.bat tests                          :: o bien
python -m pytest -q                    :: 66 pruebas
python -m pytest tests/integration/test_api_regressions.py -v
```

Fichero	Cubre
`tests/unit/test_policy_semantics.py`	Lógica ternaria y casos C08, C18, C19, C20
`tests/unit/test_contracts.py`	Inmutabilidad de referencias, bindings y revisión monótona
`tests/integration/test_m0_cases.py`	C01, C02, C03, C05, C06, C07, C15, C23, C24, registro, metas, grafo, auditoría
`tests/integration/test_p02_p15_advanced.py`	C09, C12, C21, C22, certificaciones, planner
`tests/integration/test_api_smoke.py`	Arranque, `/ready` y CRUD básico por HTTP
`tests/integration/test_api_regressions.py`	Carreras reales de C01, C07 y C09 (peticiones simultáneas), duplicados, fail-open, SSRF y las rutas nuevas de 0.2.0
`tests/integration/test_api_regressions_0_2_1.py`	Revisión 0.2.1: enteros fuera de rango, recibos que retroceden, caducidad en lecturas, reanudación tras REPLAY_GAP, epoch anterior, `event_id` divergente, propietario de certificaciones y ALLOW falsificado en el planificador, token opcional, tokens por app y certificaciones con barras en el modelo

Los tests usan siempre una base temporal (`tests/__init__.py`); no tocan la base real. Las pruebas de carrera se validaron quitando el arreglo: sin él, fallan. Las de 0.2.1 se validaron igual, una a una contra el código anterior.

21. Errores frecuentes y cómo resolverlos

Síntoma	Causa probable	Qué hacer
404 al reservar presupuesto	La cuenta no existe (en 0.2.0 consultar no la crea)	Crearla con `POST /budget/accounts`
400 `SSRFSecurityViolation` al registrar	Host público, link-local o DNS fuera de la allowlist	Usar la IP de la LAN o añadir el nombre a `AI_OS_ALLOWED_HOSTS`
Evaluar políticas devuelve DENY sin `matched_policy_refs`	No hay políticas en ese scope (denegar por defecto)	Revisar el `scope` con `GET /policies`
INDETERMINATE inesperado	Falta un dato o tiene un tipo no comparable	Mirar `unknown_inputs` y completar el contexto
403 al consumir una aprobación	Caducada, revocada, agotada o digest distinto	`GET /approvals/{id}/status`
409 `IdempotencyInProgress`	Reintento mientras la primera petición sigue en curso	Esperar `Retry-After` y repetir
Un evento no cambia la versión canónica	No es `full_state`, es de revisión menor, de un epoch anterior o espera en cuarentena a que llegue una secuencia que falta	`GET /events/quarantine` y `/events/gaps`; reenviar la secuencia perdida
400 al proponer un plan con ALLOW	La `policy_decision` no viene de `/policies/evaluate` o se modificó	Pasar la respuesta de la evaluación tal cual, con su `trace_id`
409 al registrar un recibo	El estado retrocede, ya era terminal o lo envía otro receptor	`POST /operations/status` para ver el recibo vigente

El servicio no arranca: puerto ocupado	Ya hay otra instancia en el 8000	Cerrarla o arrancar con `--port 8010`

22. Límites conocidos

> **Alcance de la autenticación**
> Con tokens por app (`AI_OS_API_TOKENS`) Core sabe quién llama y no deja que una app se haga pasar por otra. Lo que *no* controla aún es qué puede hacer cada una: los scopes y el clearance de planificador y contexto siguen viniendo en la petición. Con el token común (`AI_OS_API_TOKEN`) todas las apps son indistinguibles, y sin ninguna variable no hay comprobación. Escucha en `127.0.0.1`; no lo expongas en la red sin HTTPS delante.

- Un REPLAY_GAP se resuelve sólo cuando llega la secuencia que faltaba (Core reanuda los eventos retenidos). Si el productor la perdió de verdad, no hay todavía flujo de snapshot y nueva línea base (§6.4): la salida es abrir un `stream_epoch` nuevo.
- Las respuestas 4xx con clave de idempotencia quedan registradas: si falló por una precondición (p. ej. la meta aún no existía), el reintento necesita otra clave.
- Las metas y objetivos admiten cualquier transición de estado (también reabrir uno `completed`); la especificación no fija su ciclo de vida.
- Las políticas no se borran: se desactivan con `enabled: false`. Metas, objetivos y recursos tampoco tienen borrado.
- La "incógnita de seguridad" (C20) se detecta por el nombre del campo (*security*, *auth*).
- No hay outbox de comandos activo: la tabla `command_outbox` existe pero ningún flujo la usa todavía; tampoco hay endpoint para consultar las trazas de decisión de políticas (se guardan en `decision_traces`).
- La promoción y degradación de autonomía de P15 dependen de una `AutonomyPolicyDefinition` aceptada que aún no existe; hoy sólo se combinan techos.

23. Referencia rápida de endpoints

Método	Ruta	Para qué
GET	`/health` · `/ready`	Vida y preparación
POST · GET	`/api/v1/goals`	Crear · listar metas
GET	`/api/v1/goals/{id}` · `/objectives` · `/audit`	Meta, sus objetivos, su auditoría
PATCH	`/api/v1/goals/{id}/status`	Cambiar estado de meta
POST · GET · PATCH	`/api/v1/objectives` · `/{id}` · `/{id}/status`	Objetivos
POST · GET	`/api/v1/services` · `/{id}`	Registro de servicios
POST · GET	`/api/v1/capabilities` · `/{id}`	Capacidades
POST · GET	`/api/v1/policies`	Crear/actualizar · listar políticas
POST	`/api/v1/policies/evaluate`	Decisión consolidada
POST	`/api/v1/approvals` · `/use` · `/{id}/revoke`	Conceder · consumir · revocar
GET	`/api/v1/approvals/{id}/status`	Estado efectivo
POST	`/api/v1/budget/accounts` · `/reserve` · `/consume` · `/release`	Presupuesto
GET	`/api/v1/budget/accounts/{id}` · `/reservations/{id}`	Saldo y reservas
POST	`/api/v1/events/inbox`	Ingesta de eventos
GET	`/api/v1/events/canonical/{id}` · `/gaps` · `/quarantine`	Proyección e incidencias
POST	`/api/v1/operations/receipts` · `/status`	Restore
PUT · GET	`/api/v1/resources` · `/{id}`	Grafo de recursos
POST	`/api/v1/resources/relations`	Relaciones
PUT · GET	`/api/v1/identity-bindings` · `/{global}/{system}`	Identidades federadas
POST · GET	`/api/v1/certifications` · `/{model}/{cap}/{ver}`	Certificaciones
POST · GET	`/api/v1/context/packages` · `/{id}` · `/snapshots/{hash}`	Contexto inmutable
POST · GET	`/api/v1/planner/proposals` · `/{id}` · `/{id}/decision`	Planes y autonomía

24. Glosario

Plano de control	Parte del sistema que decide y coordina; el "plano de datos" (Broker, agentes) hace el trabajo.
Scope	Ámbito al que se aplica una política o autorización (p. ej. `broker:invoke`).
Principal	Quién actúa: usuario, servicio o agente, con su rol, scopes y clearance.
Digest	Hash SHA-256 que identifica de forma exacta un contenido o una operación.
full_state	Evento con la foto completa de un recurso; el único que mueve la proyección canónica.
Cuarentena	Evento guardado pero excluido de la proyección por hueco o contradicción.
REPLAY_GAP	Salto en la numeración de eventos de un flujo.
Techo de autonomía	Límite máximo de autonomía que impone una capa (política, certificación, usuario).
Idempotencia	Repetir una petición produce el mismo efecto que hacerla una vez.
C01…C24	Casos de aceptación de la especificación normativa 0.6.1 (§17).

Manual generado a partir del código de la versión 0.2.0 y de pruebas reales ejecutadas el 7 de octubre de 2026.